

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

GUÍA DE APLICACIÓN DE LOS TEMAS DE LA
ASIGNATURA

AL PROYECTO FIN DE CURSO (PFC)

Asignatura: Aplicaciones Distribuidas (ISR-701)

Período académico: Regular 2025-2026 SPA · Séptimo período · Modalidad grupal (3-4 integrantes)

AVISO IMPORTANTE — Carácter orientativo de esta guía. Las aplicaciones de los temas que se describen a continuación constituyen **una alternativa** de implementación, propuesta por el docente como punto de partida y referencia.

La aplicación definitiva de cada tema —tecnologías, posición en el teorema CAP, patrones, fronteras de los microservicios y decisiones arquitectónicas— **debe ser propuesta, justificada y defendida por el equipo de PFC** en sus entregas (TA-PFC-E1 a E4) y en la defensa oral. El equipo puede apartarse de estas sugerencias siempre que sustente técnicamente sus decisiones.

1. Propósito y alcance

Este documento ofrece, para cada uno de los **cuatro** proyectos de fin de curso (PFC) identificados en la asignatura, una alternativa concreta de cómo aplicar la totalidad de los temas de las cinco unidades de Aplicaciones Distribuidas. El objetivo es que cada equipo visualice de qué forma los conceptos teóricos y procedimentales del curso se materializan en su sistema, evitando que algún tema quede sin aplicación demostrable en el producto final. Los cuatro PFC considerados, extraídos del trabajo del período, son:

1. **TiendaTech** (equipo AGLS) — comercio electrónico de hardware de computadoras con asistente de compatibilidad basado en inteligencia artificial.
2. **SGA — Escuela «Provincias Unidas»** (equipo BCEL) — sistema de gestión académica (matrícula, calificaciones, asistencia y reportes).
3. **Control y Administración de Laboratorios Informáticos** (equipo FUVV) — reserva, control de acceso y monitoreo distribuido de laboratorios de cómputo.
4. **Soporte Técnico de Internet** (equipo ACC) — sistema de gestión de solicitudes (tickets) para un proveedor de servicios de Internet (ISP).

2. Mapa general de los temas de la asignatura

La asignatura se organiza en cinco unidades. La siguiente tabla resume los temas que deben quedar aplicados en el PFC y la práctica experimental sumativa asociada a cada unidad.

Unidad	Temas clave a aplicar en el PFC	Práctica
U1 — Generalidades de los SD	Transparencias ANSA; modelos y comunicación (sockets TCP, gRPC/RPC, middleware); relojes de Lamport y vectoriales; tolerancia a fallos; elección de líder (Bully/Ring); teorema CAP; coordinación y consistencia (2PC, Raft); seguridad (JWT/OAuth2/TLS); SLA y alta disponibilidad.	GA-SUM-01 Sem. 4
U3 — Arquitecturas Distribuidas	Microservicios con SRP y database-per-service; API REST + OpenAPI; API Gateway; mensajería asíncrona (RabbitMQ/Kafka); Docker y Docker Compose; Kubernetes; patrones de despliegue (blue-green, canary).	GA-SUM-02 Sem. 7
U2 — Bases de Datos Distribuidas	Fragmentación (sharding); replicación; consistencia y tolerancia a fallos en datos; clúster CockroachDB.	GA-SUM-03 Sem. 11
U4 — Cómputo Paralelo	Procesamiento distribuido con PySpark/Apache Spark; análisis de rendimiento; Ley de Amdahl.	GA-SUM-05 Sem. 14
U5 — Desarrollo en Capas y Calidad	Arquitectura N-capas; patrones GoF (Repository, Factory, Proxy, Observer, Strategy); SOLID; inyección de dependencias (IoC); CI/CD; observabilidad (Prometheus + Grafana); pruebas; calidad ISO/IEC 25010.	GA-SUM-06 Sem. 17

2.1 Relación de los temas con las cuatro entregas del PFC

El PFC se construye de forma acumulativa en cuatro entregas. Cada entrega concentra la aplicación de un conjunto de temas; el documento final (E4) los integra todos.

Entrega	Foco temático	Qué debe evidenciar
TA-PFC-E1 (Sem. 4)	U1: análisis y fundamentos	Problema cuantificado, contexto y stakeholders; diagramas C4 niveles 1-2; ≥ 3 ADR; posición CAP; transparencias ANSA aplicables; atributos ISO/IEC 25010 priorizados.
TA-PFC-E2 / GA-SUM-02 (Sem. 7)	U3: arquitectura de microservicios	Descomposición en microservicios (C4 nivel 3) con SRP y BD propia; API REST documentada (OpenAPI); API Gateway; mensajería asíncrona justificada; Dockerfile + docker-compose con demo en vivo.
TA-PFC-E3 (Sem. 13)	U2 + U4: datos y procesamiento	Esquema de fragmentación/replicación (CockroachDB); estrategia de consistencia y tolerancia a fallos en datos; job de procesamiento distribuido (PySpark) con análisis de speedup (Amdahl).
TA-PFC-E4 (Sem. 15-17)	U5: capas, calidad y cierre	Arquitectura N-capas y patrones; pipeline CI/CD; observabilidad (métricas, logs, trazas); pruebas (cobertura $\geq 70\%$, integración, carga); evaluación ISO/IEC 25010; documento LaTeX acumulativo y defensa oral.

3. PFC A — TiendaTech (e-commerce de hardware + IA)

3.1 Descripción del proyecto

TiendaTech es una tienda en línea de componentes de computación (CPU, GPU, RAM, placas base, fuentes, almacenamiento) orientada al mercado ecuatoriano. Su rasgo diferenciador es un asistente de compatibilidad basado en IA que recomienda configuraciones (builds) compatibles y dentro del presupuesto del usuario, combinando filtrado colaborativo, un motor de reglas de compatibilidad y un modelo de lenguaje con recuperación aumentada (RAG).

Equipo de referencia: AGLS (Aucatoma, Gaibor, Sánchez, Lozano). Stakeholders: clientes, administradores y la pasarela de pago externa. Roles del sistema: cliente y administrador.

3.2 Arquitectura distribuida (alternativa propuesta)

Se propone una arquitectura cliente-servidor de varios niveles (no P2P): un frontend web/móvil consume un API Gateway que enruta hacia microservicios independientes (Catálogo, Inventario, Carrito-Órdenes, Pagos, Usuarios-Autenticación, Recomendación-IA y Notificaciones). La comunicación externa es REST; la interna de alto rendimiento, gRPC; los eventos de negocio viajan por un broker de mensajería.

3.3 Aplicación de los temas, tema por tema

La tabla mapea cada concepto del curso a una alternativa concreta de aplicación en este sistema. El equipo debe confirmar, ampliar o sustituir cada celda con justificación técnica.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
UNIDAD 1 — Generalidades de los Sistemas Distribuidos	
Transparencias ANSA	Mapear al menos 4-5 de las 8: ubicación (el cliente no sabe en qué nodo está el catálogo), replicación (réplicas de catálogo/inventario), concurrencia (varios usuarios compran el mismo SKU), fallos (la caída de Pagos no derriba el Catálogo) y acceso (mismo API para web y móvil).
Modelo y comunicación Sockets TCP	Cliente-servidor de N niveles. Justificar por qué no es P2P. Frontend → API Gateway → microservicios.
gRPC / RPC	Canal de baja latencia para la reserva atómica de stock entre Inventario y Carrito-Órdenes; sirve además como base de la práctica U1 (delimitación de mensajes por longitud).
Middleware	Comunicación interna de alto rendimiento entre Órdenes ↔ Inventario ↔ Recomendación-IA, con contratos .proto. El motor de compatibilidad se expone como servicio RPC.
Relojes de Lamport / vectoriales	API Gateway como punto único de entrada; broker (RabbitMQ/Kafka) para eventos de dominio como OrdenCreada y StockBajo.
Tolerancia a fallos	Ordenar de forma total los eventos concurrentes sobre el stock de un mismo SKU (reservas simultáneas) entre nodos de Inventario; relojes vectoriales para reconciliar el carrito abierto en varios dispositivos.
Elección de líder (Bully/Ring)	Heartbeats entre réplicas de Pagos e Inventario; circuit breaker hacia la pasarela externa. Fallo más probable: omisión/temporización en la pasarela de pago.
	Elegir el nodo coordinador que serializa la asignación de números de orden únicos y procesa la cola de órdenes; reelección automática si cae el líder.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Teorema CAP	Por agregado: CP en stock/órdenes (no vender lo inexistente) y AP en catálogo/reseñas (priorizar disponibilidad). Justificar la posición por dominio.
Coordinación y consistencia (2PC/Raft)	Confirmar atómicamente «descontar stock + cobrar + crear orden» mediante 2PC o, alternativamente, una saga con compensación (discutir el trade-off). Raft como consenso del clúster de datos.
Seguridad	JWT para sesiones de cliente, OAuth2 para login social, TLS en todos los canales, RBAC (cliente vs. administrador) y tokenización de datos de pago.
SLA y alta disponibilidad	Definir 99,5 % de disponibilidad del catálogo; réplicas activas y degradación elegante del recomendador IA bajo carga.
UNIDAD 3 — Arquitecturas Distribuidas	
Microservicios (SRP + BD propia)	Catálogo, Inventario, Carrito-Órdenes, Pagos, Usuarios-Auth, Recomendación-IA y Notificaciones, cada uno con su base de datos.
API REST + OpenAPI	Endpoints públicos de catálogo, carrito y órdenes documentados con OpenAPI 3.0.
API Gateway	Enrutamiento, autenticación centralizada y rate limiting (en especial al endpoint de recomendación IA).
Mensajería asíncrona	RabbitMQ para colas de trabajo (OrdenCreada → Facturación, Notificaciones) y, opcionalmente, Kafka para el stream de eventos de navegación que alimenta la IA. Justificar la elección.
Docker	Cada microservicio en su contenedor con build multi-stage.
Orquestación	Docker Compose en desarrollo; Kubernetes para escalar bajo demanda el servicio de Recomendación-IA.
Patrones de despliegue	Canary para liberar nuevas versiones del modelo de recomendación a un porcentaje de usuarios antes del despliegue total.
UNIDAD 2 — Bases de Datos Distribuidas	
Fragmentación (sharding)	Particionar el catálogo por categoría/marca y las órdenes por fecha/región.
Replicación y consistencia	CockroachDB para inventario y órdenes con consistencia serializable y réplicas geográficas.
Tolerancia a fallos en datos	Recuperación automática ante la caída de un nodo del clúster, sin pérdida de transacciones confirmadas.
UNIDAD 4 — Cómputo Paralelo	
Procesamiento distribuido (PySpark)	Procesar el histórico de navegación y compras para alimentar el filtrado colaborativo y generar reportes de ventas/tendencias.
Ley de Amdahl	Medir el speedup del job de recomendación al añadir workers e identificar la fracción secuencial del cálculo.
UNIDAD 5 — Desarrollo en Capas y Calidad	
N-capas + SOLID	Cada microservicio en capas presentación/aplicación/dominio/infraestructura, respetando responsabilidades únicas.
Patrones GoF	Repository (catálogo), Strategy (métodos de pago y algoritmos de recomendación), Factory (creación de estrategias de pago), Observer (eventos de stock) y Proxy (caché del catálogo).

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Inyección de dependencias (IoC)	Contenedor IoC de Spring Boot para desacoplar servicios y repositorios.
CI/CD	GitHub Actions: pruebas automáticas, build de imagen Docker y análisis de calidad de código.
Observabilidad	Métricas de latencia del checkout en Prometheus/Grafana, logs estructurados JSON y trazas distribuidas de una orden a través de los servicios.
Pruebas	Unitarias del motor de compatibilidad (cobertura $\geq 70\%$), integración del checkout y carga con Locust simulando picos tipo «Black Friday».
Calidad ISO/IEC 25010	Priorizar idoneidad funcional (recomendación correcta), eficiencia de desempeño (latencia de checkout), seguridad (pagos) y mantenibilidad.

3.4 Notas de diseño

- Posición CAP: conviene argumentarla por agregado del dominio, no para todo el sistema. El inventario/órdenes exige consistencia (CP); el catálogo y reseñas toleran disponibilidad eventual (AP).
- Modelo de fallos: el punto crítico es la integración con la pasarela de pago externa (fallos de omisión y temporización). Diseñar idempotencia y reintentos.
- La IA es el diferenciador: el filtrado colaborativo justifica el uso de PySpark (U4); el motor de reglas de compatibilidad y el RAG conectan con los patrones de la U5.
- Decisión clave a defender: 2PC frente a saga para la transacción de compra. Ambas son válidas; el equipo elige y sustenta.

3.5 Preguntas que el equipo debe poder responder en la defensa

1. ¿Dónde se ubica TiendaTech en el teorema CAP y por qué la respuesta es distinta para el inventario y para el catálogo?
2. ¿Por qué eligieron 2PC (o una saga) para la transacción de compra y qué pasa si falla la pasarela de pago a mitad del proceso?
3. ¿Qué fracción del job de recomendación es paralelizable y qué speedup esperan según la Ley de Amdahl?

4. PFC B — SGA, Escuela «Provincias Unidas»

4.1 Descripción del proyecto

El SGA es un Sistema de Gestión Académica para una unidad educativa (aproximadamente 344 estudiantes y 14 docentes). Cubre matrícula, gestión de estudiantes y docentes, registro de calificaciones, control de asistencia, horarios y generación de reportes.

Equipo de referencia: BCEL (Bedón, Castro, Emanuel, Luna). Stakeholders: estudiantes, representantes/apoderados, docentes, personal administrativo y directivos. Punto sensible: el sistema gestiona datos de menores de edad.

4.2 Arquitectura distribuida (alternativa propuesta)

Arquitectura cliente-servidor de varios niveles con portales diferenciados (administrativo, docente y de representantes) sobre un API Gateway. Microservicios sugeridos: Matrícula, Estudiantes, Docentes, Calificaciones, Asistencia, Horarios, Reportes, Autenticación y Notificaciones. Comunicación externa REST; interna gRPC; eventos de dominio por broker.

4.3 Aplicación de los temas, tema por tema

La tabla mapea cada concepto del curso a una alternativa concreta de aplicación en este sistema. El equipo debe confirmar, ampliar o sustituir cada celda con justificación técnica.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
UNIDAD 1 — Generalidades de los Sistemas Distribuidos	
Transparencias ANSA	Ubicación (consultar notas sin saber el nodo), concurrencia (varios docentes cargando calificaciones a la vez), replicación (réplicas de matrícula) y fallos (la caída de Reportes no afecta la carga de notas).
Modelo y comunicación Sockets TCP	Cliente-servidor de N niveles con tres portales. Justificar la centralización frente a esquemas distribuidos por sede.
gRPC / RPC	Servicio de sincronización de asistencia desde dispositivos del aula (lectores/tablets) hacia el nodo central; base de la práctica U1.
Middleware	Comunicación interna Matrícula ↔ Calificaciones ↔ Reportes para consolidar promedios y actas.
Relojes de Lamport / vectoriales	API Gateway como punto único; broker para eventos NotaPublicada y MatriculaConfirmada.
Tolerancia a fallos	Ordenar de forma total y auditable los eventos de carga y modificación de calificaciones concurrentes (quién cambió qué nota y en qué orden); relojes vectoriales para conciliar ediciones realizadas sin conexión.
Elección de líder (Bully/Ring)	Heartbeats entre réplicas de Calificaciones; fallo más probable: omisión en la sincronización móvil de asistencia.
Teorema CAP	Elegir el nodo que ejecuta el cierre de período y consolida actas/rankings, garantizando que solo uno los genere.
Coordinación y consistencia (2PC/Raft)	CP para matrícula y notas (una matrícula no puede duplicarse; una nota debe ser única y consistente); AP tolerable para consultas de horarios.
	2PC para la matrícula atómica «reservar cupo + registrar estudiante + generar deuda»; Raft como consenso del clúster de datos.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Seguridad	JWT, RBAC con cuatro roles (estudiante, representante, docente, administrador), TLS y controles reforzados por tratarse de datos de menores (LOPDP).
SLA y alta disponibilidad	Disponibilidad $\geq 99,5$ % durante el período de matrícula, que es el pico de carga del sistema.
UNIDAD 3 — Arquitecturas Distribuidas	
Microservicios (SRP + BD propia)	Matrícula, Estudiantes, Docentes, Calificaciones, Asistencia, Horarios, Reportes, Auth y Notificaciones, cada uno con su base de datos.
API REST + OpenAPI	Endpoints de matrícula, calificaciones y reportes documentados con OpenAPI 3.0.
API Gateway	Enrutamiento, autenticación y autorización por rol antes de llegar a cada microservicio.
Mensajería asíncrona	RabbitMQ para NotaPublicada → Notificaciones a representantes y MatriculaConfirmada → Facturación/Reportes.
Docker	Contenedor por microservicio con build multi-stage.
Orquestación	Docker Compose en desarrollo; Kubernetes para escalar Matrícula y Calificaciones en sus picos.
Patrones de despliegue	Blue-green para actualizar sin interrumpir el período de matrícula.
UNIDAD 2 — Bases de Datos Distribuidas	
Fragmentación (sharding)	Particionar por período lectivo y/o por grado/sección.
Replicación y consistencia	CockroachDB para Estudiantes y Calificaciones con consistencia serializable.
Tolerancia a fallos en datos	Réplicas multi-nodo con recuperación automática para no perder calificaciones confirmadas.
UNIDAD 4 — Cómputo Paralelo	
Procesamiento distribuido (PySpark)	Análisis masivo de rendimiento académico: promedios por curso, detección temprana de riesgo de deserción y estadísticas institucionales.
Ley de Amdahl	Speedup del cálculo de estadísticas institucionales al variar el número de workers.
UNIDAD 5 — Desarrollo en Capas y Calidad	
N-capas + SOLID	Cada microservicio en capas, con separación clara de responsabilidades.
Patrones GoF	Repository (estudiantes), Strategy (distintos esquemas de calificación por nivel), Observer (publicación de notas → notificación) y Factory (generación de reportes en varios formatos).
Inyección de dependencias (IoC)	Contenedor IoC de Spring Boot.
CI/CD	GitHub Actions con pruebas, build Docker y análisis de calidad.
Observabilidad	Métricas, logs estructurados y trazas; tablero Grafana con el estado del período de matrícula.
Pruebas	Unitarias (≥ 70 %), integración del flujo de matrícula y carga con Locust simulando matrícula simultánea masiva.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Calidad ISO/IEC 25010	Priorizar fiabilidad, idoneidad funcional, seguridad (datos de menores) y usabilidad de los portales.

4.4 Notas de diseño

- Posición CAP: el dominio académico es marcadamente CP para matrícula y notas; la integridad prima sobre la disponibilidad. Documentar el modelo de consistencia elegido.
- El período de matrícula es el escenario de carga crítica; toda la estrategia de escalado, SLA y pruebas de carga debe girar en torno a ese pico.
- Seguridad reforzada por tratarse de datos de menores: la auditoría de cambios de calificaciones (apoyada en relojes de Lamport) es un requisito, no un extra.
- Las estadísticas institucionales y la detección de deserción son la carga analítica que justifica PySpark (U4).

4.5 Preguntas que el equipo debe poder responder en la defensa

1. ¿Por qué el SGA se posiciona como CP en el teorema CAP para matrícula y calificaciones?
2. ¿Cómo garantizan, con relojes lógicos, una auditoría confiable del orden de modificación de calificaciones entre nodos?
3. ¿Qué microservicios escalan durante el período de matrícula y cómo lo demuestran con sus pruebas de carga?

5. PFC C — Control y Administración de Laboratorios Informáticos

5.1 Descripción del proyecto

Sistema distribuido para el control y la administración de los laboratorios de cómputo de una institución (varios laboratorios y posiblemente varios edificios/campus). Gestiona la reserva de equipos y horarios, el control de acceso, el monitoreo en tiempo real del estado de las máquinas (libre, en uso, en falla), el inventario de hardware/software y los tickets de mantenimiento. Su pregunta de investigación central gira en torno a mejorar el acceso concurrente a los recursos del laboratorio.

Equipo de referencia: FUVV (Farinango, Urbina, Villamarín, Vinuesa). Stakeholders: estudiantes, docentes, técnicos de laboratorio y administradores. Entidades clave: laboratorio, equipo/máquina, reserva, sesión de uso, incidencia de mantenimiento.

5.2 Arquitectura distribuida (alternativa propuesta)

Arquitectura cliente-servidor con un agente en cada laboratorio (o por máquina) que reporta el estado al servidor central a través de un API Gateway. Microservicios sugeridos: Reservas, Laboratorios-Equipos, Monitoreo, Control-Acceso, Mantenimiento, Autenticación y Notificaciones. Comunicación externa REST; interna gRPC; eventos por broker. El acceso concurrente a un mismo equipo/franja es el problema distribuido central.

5.3 Aplicación de los temas, tema por tema

La tabla mapea cada concepto del curso a una alternativa concreta de aplicación en este sistema. El equipo debe confirmar, ampliar o sustituir cada celda con justificación técnica.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
UNIDAD 1 — Generalidades de los Sistemas Distribuidos	
Transparencias ANSA	Ubicación (reservar un equipo sin saber en qué laboratorio/nodo está), concurrencia (varios usuarios reservan el mismo equipo/franja a la vez — núcleo del proyecto), replicación y fallos (la caída del monitoreo de un laboratorio no impide reservar en otro).
Modelo y comunicación Sockets TCP	Cliente-servidor con agentes por laboratorio que reportan el estado de las máquinas al servidor central.
gRPC / RPC	Agente en cada laboratorio/PC que envía por TCP el estado (libre, en uso, en falla) y un heartbeat al servidor; base de la práctica U1.
Middleware	Comunicación interna de alta frecuencia Reservas ↔ Monitoreo ↔ Control-Acceso.
Relojes de Lamport / vectoriales	API Gateway como punto único; broker (RabbitMQ) para eventos EquipoReservado y EquipoEnFalla.
Tolerancia a fallos	Ordenar de forma total las solicitudes de reserva concurrentes sobre el mismo equipo y franja (resolver «quién reservó primero»); relojes vectoriales para conciliar reservas originadas en distintos nodos/laboratorios.
Elección de líder (Bully/Ring)	Heartbeats de los agentes de laboratorio para detectar máquinas o nodos caídos; fallo más probable: omisión/temporización en los reportes de estado de los agentes.
	Elegir el nodo coordinador que arbitra las reservas (otorga la franja a una sola solicitud) y consolida el estado global de los laboratorios; reelección si cae.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Teorema CAP	CP en reservas (no otorgar el mismo equipo a dos personas: consistencia fuerte); AP tolerable en el monitoreo y las consultas de estado.
Coordinación y consistencia (2PC/Raft)	2PC para confirmar atómicamente «reservar equipo + bloquear franja + registrar usuario»; Raft como consenso del clúster de datos.
Seguridad	JWT, RBAC (estudiante, docente, técnico, administrador), TLS y control de acceso lógico al laboratorio enlazado con la reserva vigente.
SLA y alta disponibilidad	Disponibilidad del módulo de reservas en horas pico y tiempo de respuesta máximo del monitoreo.
UNIDAD 3 — Arquitecturas Distribuidas	
Microservicios (SRP + BD propia)	Reservas, Laboratorios-Equipos, Monitoreo, Control-Acceso, Mantenimiento, Auth y Notificaciones, cada uno con su base de datos.
API REST + OpenAPI	Endpoints de reserva, consulta de disponibilidad y estado de laboratorios documentados con OpenAPI 3.0.
API Gateway	Enrutamiento, autenticación y rate limiting; único punto de entrada.
Mensajería asíncrona	RabbitMQ para EquipoEnFalla → Mantenimiento + Notificación y ReservaConfirmada → Control-Acceso.
Docker	Contenedor por microservicio con build multi-stage; imagen del agente de laboratorio.
Orquestación	Docker Compose en desarrollo; Kubernetes para escalar Reservas y Monitoreo en horas pico.
Patrones de despliegue	Blue-green o canary para actualizar sin interrumpir el servicio de reservas.
UNIDAD 2 — Bases de Datos Distribuidas	
Fragmentación (sharding)	Particionar por laboratorio/edificio/campus.
Replicación y consistencia	CockroachDB para Reservas y Equipos con consistencia fuerte donde se evita la doble reserva.
Tolerancia a fallos en datos	Recuperación automática ante caída de nodos; no perder reservas confirmadas.
UNIDAD 4 — Cómputo Paralelo	
Procesamiento distribuido (PySpark)	Análisis del uso de laboratorios: ocupación por hora y equipo, software más utilizado, detección de equipos subutilizados o con fallas recurrentes a partir de los logs de uso.
Ley de Amdahl	Speedup del procesamiento de los logs de uso al escalar workers.
UNIDAD 5 — Desarrollo en Capas y Calidad	
N-capas + SOLID	Cada microservicio en capas, con responsabilidades únicas.
Patrones GoF	State (estado del equipo: libre/ocupado/mantenimiento), Observer (cambio de estado → monitoreo/notificación), Strategy (políticas de reserva y prioridad), Repository y Proxy.
Inyección de dependencias (IoC)	Contenedor IoC de Spring Boot.
CI/CD	GitHub Actions con pruebas, build Docker y análisis de calidad.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Observabilidad	Tablero Grafana con el estado en vivo de todos los laboratorios; métricas, logs estructurados y trazas.
Pruebas	Unitarias del motor de reservas ($\geq 70\%$), integración y carga con Locust simulando reservas concurrentes masivas (escenario crítico del proyecto).
Calidad ISO/IEC 25010	Priorizar fiabilidad, eficiencia de desempeño (acceso concurrente, alineado con su RQ), seguridad y mantenibilidad.

5.4 Notas de diseño

- El acceso concurrente es la pregunta de investigación central: los relojes de Lamport para ordenar reservas y la posición CP del módulo de reservas son el corazón del diseño y deben demostrarse, no solo enunciarse.
- Modelo de fallos: el punto débil son los reportes de estado de los agentes de laboratorio (omisión/temporización); diseñar detección por heartbeat y reconciliación.
- La prueba de carga decisiva (U5) es la de reservas concurrentes simultáneas sobre el mismo recurso: es donde se valida la RQ.
- Sugerencia respecto al estado del arte: anclar las referencias al dominio (gestión académica, reservas, laboratorios, sistemas educativos distribuidos) y no a control industrial o smart grids.

5.5 Preguntas que el equipo debe poder responder en la defensa

1. ¿Cómo resuelven, con relojes lógicos, dos reservas concurrentes sobre el mismo equipo y la misma franja?
2. ¿Por qué el módulo de reservas se posiciona como CP y el de monitoreo puede ser AP?
3. ¿Qué métrica de acceso concurrente (su RQ) miden y con qué prueba de carga la validan?

6. PFC D — Soporte Técnico de Internet (gestión de tickets de un ISP)

6.1 Descripción del proyecto

Sistema de gestión de solicitudes (tickets) para un proveedor de servicios de Internet. Los abonados reportan incidencias (caídas, lentitud, fallas de equipo), el sistema las clasifica por prioridad, asigna técnicos y agenda visitas, y mide el cumplimiento de los acuerdos de nivel de servicio (SLA). Es el proyecto en el que el SLA y la alta disponibilidad son el corazón del dominio.

Equipo de referencia: ACC (Pacheco, Carpio, Álvarez, Cando). Stakeholders: abonados/-clientes, técnicos de campo, supervisores/despachadores y administración. Entidades clave: ticket, cliente, técnico, agenda/visita, diagnóstico, SLA.

6.2 Arquitectura distribuida (alternativa propuesta)

Arquitectura cliente-servidor con portal del cliente y aplicación del técnico, sobre un API Gateway. Microservicios sugeridos: Tickets, Clientes, Técnicos-Agenda, Diagnóstico, Notificaciones, Autenticación y Reportes-SLA. El ciclo de vida del ticket se modela como una máquina de estados. Comunicación externa REST; interna gRPC; eventos por broker (RabbitMQ), con posibilidad de un stream (Kafka) para telemetría de red.

6.3 Aplicación de los temas, tema por tema

La tabla mapea cada concepto del curso a una alternativa concreta de aplicación en este sistema. El equipo debe confirmar, ampliar o sustituir cada celda con justificación técnica.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
UNIDAD 1 — Generalidades de los Sistemas Distribuidos	
Transparencias ANSA	Ubicación (el cliente abre un ticket sin saber qué nodo lo atiende), concurrencia (varios técnicos sobre un incidente masivo de una zona), fallos y replicación de la cola de tickets.
Modelo y comunicación Sockets TCP	Cliente-servidor con dos frentes (portal cliente y app de campo). Justificar frente a alternativas.
gRPC / RPC	Canal de telemetría/heartbeat desde los equipos de red del abonado (CPE) que reportan estado y caídas; base de la práctica U1.
Middleware Relojes de Lamport / vectoriales	Comunicación interna de alta frecuencia Tickets ↔ Diagnóstico ↔ Técnicos-Agenda.
Tolerancia a fallos	API Gateway y broker RabbitMQ para el flujo de trabajo de tickets.
Elección de líder (Bully/Ring)	Ordenar de forma causal el ciclo de vida del ticket (creado → asignado → en proceso → resuelto) cuando los eventos llegan desde canales distintos (web, app, call center); relojes vectoriales para detectar actualizaciones concurrentes del mismo ticket.
Teorema CAP	Elegir el nodo despachador que asigna técnicos y correlaciona incidencias masivas de una misma zona; reelección si cae.
Coordinación y consistencia (2PC/Raft)	AP en la creación/estado de tickets (el abonado debe poder reportar una falla aun con partición, reconciliando después); CP para SLA y facturación.
	2PC para «asignar técnico + reservar franja de agenda + actualizar ticket»; Raft para el consenso del clúster.

Concepto (Unidad · Tema)	Alternativa de aplicación en este PFC
Seguridad	JWT, RBAC (cliente, técnico, supervisor, administrador), TLS y registro de auditoría de cada cambio de estado.
SLA y alta disponibilidad	Núcleo del proyecto: definir tiempos de respuesta/resolución por prioridad, medir su cumplimiento y asegurar alta disponibilidad del portal de reporte de fallas.
UNIDAD 3 — Arquitecturas Distribuidas	
Microservicios (SRP + BD propia)	Tickets, Clientes, Técnicos-Agenda, Diagnóstico, Reportes-SLA, Auth y Notificaciones, cada uno con su base de datos.
API REST + OpenAPI	Endpoints de creación y seguimiento de tickets documentados con OpenAPI 3.0.
API Gateway	Enrutamiento, autenticación y rate limiting; único punto de entrada.
Mensajería asíncrona	RabbitMQ para TicketCreado → Despacho + Notificación, con enrutamiento por prioridad (colas de trabajo). Kafka, opcional, para el stream de telemetría de red. Justificar RabbitMQ frente a Kafka según el flujo.
Docker	Contenedor por microservicio con build multi-stage.
Orquestación	Docker Compose en desarrollo; Kubernetes para absorber picos de incidencias masivas.
Patrones de despliegue	Canary o blue-green para actualizar sin afectar la recepción de incidencias.
UNIDAD 2 — Bases de Datos Distribuidas	
Fragmentación (sharding)	Particionar tickets por zona geográfica y/o por fecha.
Replicación y consistencia	CockroachDB para Tickets y SLA con réplicas; consistencia fuerte donde la factura/SLA lo exige.
Tolerancia a fallos en datos	Recuperación automática ante caída de nodos; no perder tickets registrados.
UNIDAD 4 — Cómputo Paralelo	
Procesamiento distribuido (PySpark)	Análisis de telemetría de red e histórico de incidencias para detectar zonas problemáticas, calcular el MTTR y anticipar fallas masivas.
Ley de Amdahl	Speedup del análisis de logs de red al escalar workers.
UNIDAD 5 — Desarrollo en Capas y Calidad	
N-capas + SOLID	Cada microservicio en capas, con responsabilidades únicas.
Patrones GoF	State (máquina de estados del ticket), Strategy (políticas de priorización y cálculo de SLA), Observer (cambios de estado → notificación), Repository y Proxy.
Inyección de dependencias (IoC)	Contenedor IoC de Spring Boot.
CI/CD	GitHub Actions con pruebas, build Docker y análisis de calidad.
Observabilidad	Métricas de cumplimiento de SLA, logs estructurados y trazas del recorrido de un ticket; tablero Grafana de incidencias en tiempo real.
Pruebas	Unitarias del motor de SLA ($\geq 70\%$), integración del flujo de despacho y carga con Locust simulando una caída masiva.
Calidad ISO/IEC 25010	Priorizar fiabilidad, eficiencia de desempeño, seguridad y mantenibilidad.

6.4 Notas de diseño

- Posición CAP: a diferencia de los otros tres PFC, aquí conviene priorizar disponibilidad (AP) en la recepción de incidencias — el abonado debe poder reportar siempre — reconciliando el estado más tarde; reservar la consistencia fuerte para SLA y facturación.
- El SLA no es un atributo secundario: es el eje del dominio. El patrón State para el ciclo de vida del ticket, junto con relojes de Lamport, mantiene la coherencia entre canales.
- Justificación de mensajería: RabbitMQ encaja con colas de trabajo y enrutamiento por prioridad; Kafka con el stream de telemetría. Es un buen caso para contrastar ambos.
- La telemetría de red y el cálculo de MTTR son la carga analítica que justifica PySpark (U4).

6.5 Preguntas que el equipo debe poder responder en la defensa

1. ¿Por qué la creación de tickets se posiciona como AP y el SLA/facturación como CP en el teorema CAP?
2. ¿Por qué eligieron RabbitMQ para el despacho y, en su caso, Kafka para la telemetría? ¿Qué los diferencia para cada flujo?
3. ¿Cómo mantienen consistente el ciclo de vida del ticket (patrón State + relojes lógicos) cuando los eventos llegan por web, app y call center?

7. Recomendaciones transversales y criterios de evaluación

Con independencia del proyecto, todos los equipos deben cumplir los siguientes requisitos, alineados con los instrumentos de evaluación de la asignatura:

- **Documentación en LaTeX con calidad académica:** el documento del PFC debe compilar sin errores, con código en `lstlisting`, figuras con `\caption` y `\label` referenciadas, tablas en `booktabs` y bibliografía IEEE verificada con DOI. Un documento que no sea LaTeX (por ejemplo, Word exportado a PDF) recibe cero en los criterios de código y formato.
- **Repositorio GitHub con historial real:** cada integrante debe tener commits significativos. La ausencia de commits de un integrante conlleva penalización individual.
- **Declaración de uso de IA:** obligatoria; su ausencia se penaliza de forma uniforme.
- **Verificación de referencias:** los DOI deben resolver y los autores/títulos deben ser reales y *pertinentes al dominio*. Las referencias fabricadas o ajenas al tema constituyen una debilidad de integridad o de fundamentación.
- **Evaluación de calidad con ISO/IEC 25010:2023:** priorizar y medir, con métricas reales, los atributos de calidad pertinentes a cada dominio.
- **Rúbricas aplicables:** TA-PFC-E1 (rúbrica científica de 7 dimensiones para la propuesta), las rúbricas de práctica experimental por unidad (PE-U1 a PE-U5) y la rúbrica de exposición para los dos cortes.

8. Nota final

Esta guía es un instrumento de apoyo. Reitera el aviso inicial: las aplicaciones aquí descritas son **una alternativa de partida**; la aplicación definitiva de cada tema corresponde al equipo de PFC, que debe proponerla, justificarla técnicamente y defenderla. Dos equipos con el mismo proyecto pueden llegar a decisiones distintas y ambas ser correctas si están bien fundamentadas.